Part 1 线性结构

Part 1 线性结构
- 第一讲 线性表
  - 约瑟夫问题
  - 线性表的定义
  - 线性表的逻辑结构
  - 线性表的存储结构
  - 线性表的运算
  - 顺序表
  - 列表list的其他方法
  - 链表
  - 链表上的运算
  - 单循环链表
  - 双链表
  - 双向循环链表
  - 链表的边界条件
  - 思考：有无表头的区别
  - 顺序表、链表的比较
  - Python中的线性表

第一讲 线性表

约瑟夫问题

> **背景**
> Josephus在自己的日记中写道：在罗马人占领乔塔帕特后，39 个犹太人与 Josephus及他的朋友躲到一个洞中，39个犹太人决定宁愿死也不要被敌人抓到， 于是决定了一个自杀方式。
> 自杀方式的约定：首先让41个人排成一个圆圈，由第1个人开始报数，每报数到第3人该人就必须自杀，然后再由下一个重新报数，直到所有人都自杀身亡为止。

> 结果：Josephus要他的朋友先假装遵从，他将朋友与自己安排在第16个与第31个 位置，于是逃过了这场死亡游戏。
> 后传：约瑟夫斯说服了他的朋友，他们将向罗马军队投降，不再自杀。

约瑟夫问题解答算法

输入：
n => 参与游戏的人数，每个人的信息[编号] 即线性表
s => 开始的人
m => 单次计数
输出：
按出列次序得到的人员信息的[编号]序列

线性表的定义

> **线性表**简称**表**，是零个或多个元素的有穷序列，可表示成
>
> $$B=(K,R),K=\{a_0,a_1,\cdots,a_{n-1}\},R=\{r\}$$
>
> 其特点是：
>
> 1. 有唯一的开始结点（表头），它没有前驱，有唯一的直接后继
> 2. 有唯一的中止结点（表尾），它没有后继，有唯一的直接前驱
> 3. 其他结点皆称为内部结点，每个内部结点都有且仅有一个唯一的直接前驱/后继
> 4. 前驱后继关系$r=<a_i,a_{i+1}>$，具有反对称性和传递性

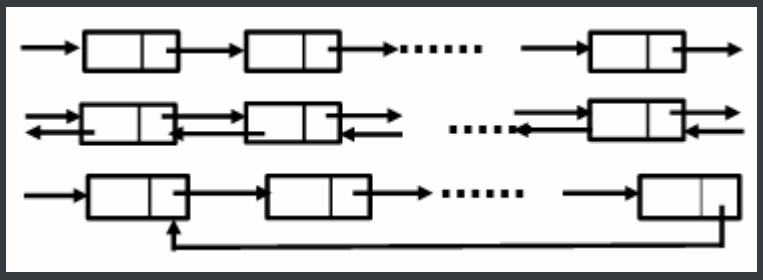
线性表的逻辑结构

- **均匀性**：对于同一线性表的个数据元素必定具有相同的数据结构类型和长度
- **有序性**：各数据元素在线性表中都有自己的位置，且数据元素之间的相对位置是线性的

线性表的存储结构

- **顺序表**：按索引值从小到大存放在一片相邻的区域，紧凑结构，存储密度为1

- **链表**：每个元素独立储存，元素之间通过“指针”连接



线性表的运算

- 建立线性表
- 清除线性表
- 插入一个新元素
- 删除某个元素
- 修改某个元素
- 排序
- 检索
- ...

顺序表

定义：采用定长的一维数组存储的线性表

主要特性：

- 元素顺序存储在连续存储空间（数组）
- 使用常数作为向量长度
- 通过下标即可指定位置（线性表中任意数据元素都可以随机存取）

顺序表（类型）定义：

```
class SeqList():
    def __init__(self,length:int):
        if length <= 0:
            raise ValueError(length)
        self.max_length = length
        self.length = 0
        self.element = [None]*length
```

```
seqlist = SeqList(10)
'''
创建顺序表对象seqlist；
则可以访问：
seqlist.element
seqlist.length
'''
```

顺序表上的运算

```
List(length:int)        #创建定长的空顺序表
insert(index:int,x)     #插入一个元素
delete(index:int)       #删除某个元素
__setitem__(index:int,x)    #实现下标访问赋值：seqlist[index] = x
__getitem__(index:int)  #实现下标访问：x = seqlist[index]
next(index:int)         #查找某个元素的后继元素
previous(index:int)     #查找某个元素的前驱元素
isEmpty()               #判别一个线性表是否为空表
clear()                 #清空整个顺序表
...
```

```
def __getitem__(self,index:int):
    if index >= 0 and index < self.length:
        return self.element[index]
    else:
        raise IndexError(index)

def __setitem__(self,index:int,value):
    if index >= 0 and index < self.length:
        self.element[index] = value
    else:
        raise IndexError(index)

def insert(self,index:int,value) -> None:
    if index < 0 or index > self.length:
        raise IndexError(index)
```

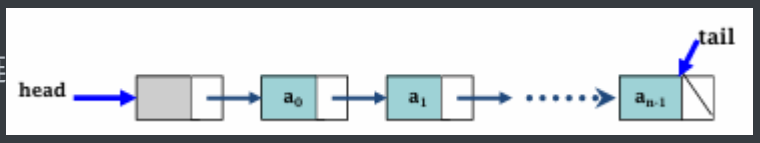
```
        if self.length == self.max_length:
            raise Exception("The sequential list is full.")
        #倒序遍历，将该元素后的元素右移
        for i in range(self.length,index,-1):
            self.element[i] = self.element[i-1]
        #插入该元素
        self.element[index] = value
        self.length += 1

    def delete(self,index:int):
        if index < 0 or index >= self.length:
            raise IndexError(index)
        #正序遍历，将该元素后的元素前移
        for i in range(index,self.length-1):
            self.element[i] = self.element[i+1]
        self.length -= 1
```

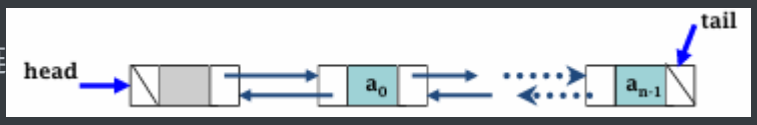
列表list的其他方法

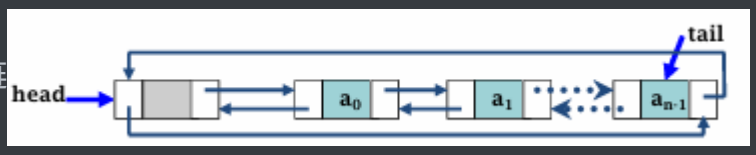
方法名称	使用例子	说明
append	alist.append(item)	列表末尾添加元素
insert	alist.insert(i,item)	列表中i位置插入元素
pop	alist.pop()	删除最后一个元素，并返回其值
pop	alist.pop(i)	删除第i个元素，并返回其值
sort	alist.sort()	将表中元素按升序排列
reverse	alist.reverse()	将表中元素按降序排列
del	del alist[i]	删除第i个元素
index	alist.index(item)	找到item的首次出现位置
count	alist.count(item)	返回item在列表中出现的次数
remove	alist.remove(item)	将item的首次出现删除

链表

分类（根据链接方式和指针多寡）：

- 单链 

- 双链 

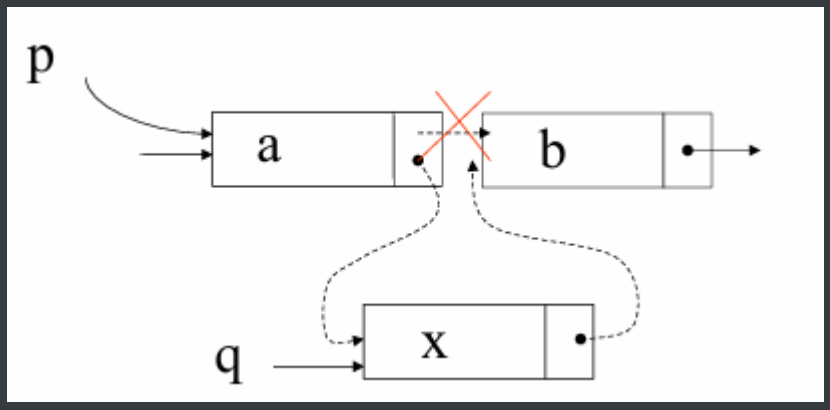
- 循环链 


```
class LinkList():
    #建立带有头结点的空链表
    def __init__(self):
        self.head:LinkNode = None
        self.length = 0
class LinkNode():
    def __init__(self,node_data):
        self.data = node_data
        self.next:LinkNode = None
```

链表上的运算

单链表插入运算的实现

在plist带头结点的单链表下标为i（第i+1个）的结点前插入一个值为x的元素，并返回插入成功与否的标志。

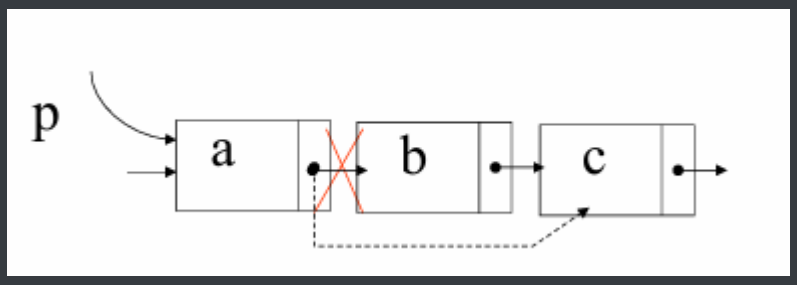


代码实现：

```
    def insert(self,index:int,value):
        if index < 0 or index > self.length:
            raise IndexError(index)
        node = LinkNode(value)
        #寻找待插入结点的前驱
        cur_node,position = self.head,0      #查找第i个位置的前驱
        while position < index:
            cur_node = cur_node.next
            position += 1
        node.next = cur_node.next            #先与后半部分建立连接
        cur_node.next = node                 #在与前半部分建立连接
        self.length += 1
```

单链表删除运算的实现

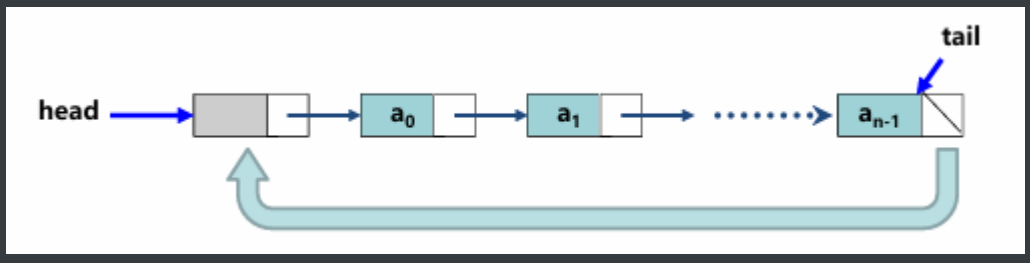
在plist带头结点的单链表中删除第一个元素为x的元素，并返回插入成功与否的标志。



代码实现：

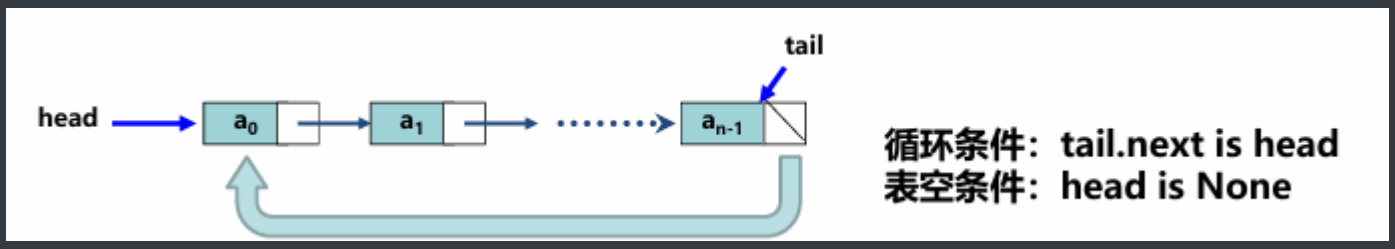
```python
    def delete(self,value):
        if self.head.next is None or value is None:
            return
        cur_node:LinkNode = self.head
        #查找待删除结点的前驱
        while cur_node is not None and cur_node.next.data != value:
            cur_node = cur_node.next
        if cur_node is None:
            return -1
        cur_node.next = cur_node.next.next
        return 0
```

单循环链表

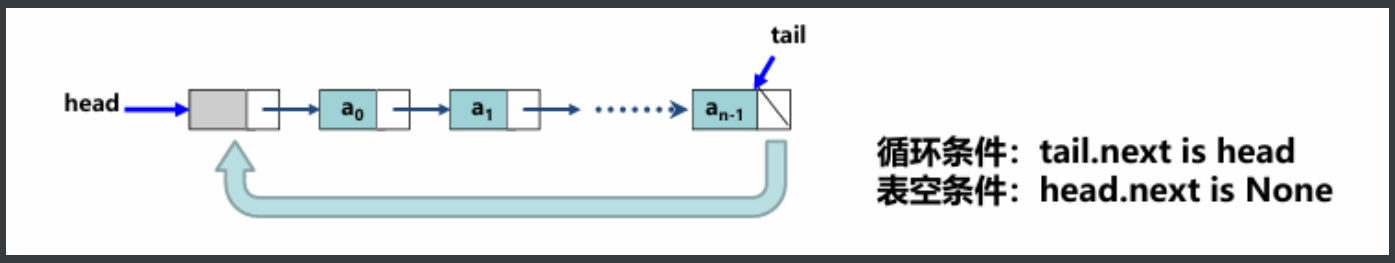


建立：将最后一个结点的指针项指向第一个结点（或头结点），构成一个循环链
优点：从任意一个结点出发，可以访问表中任何一个结点元素

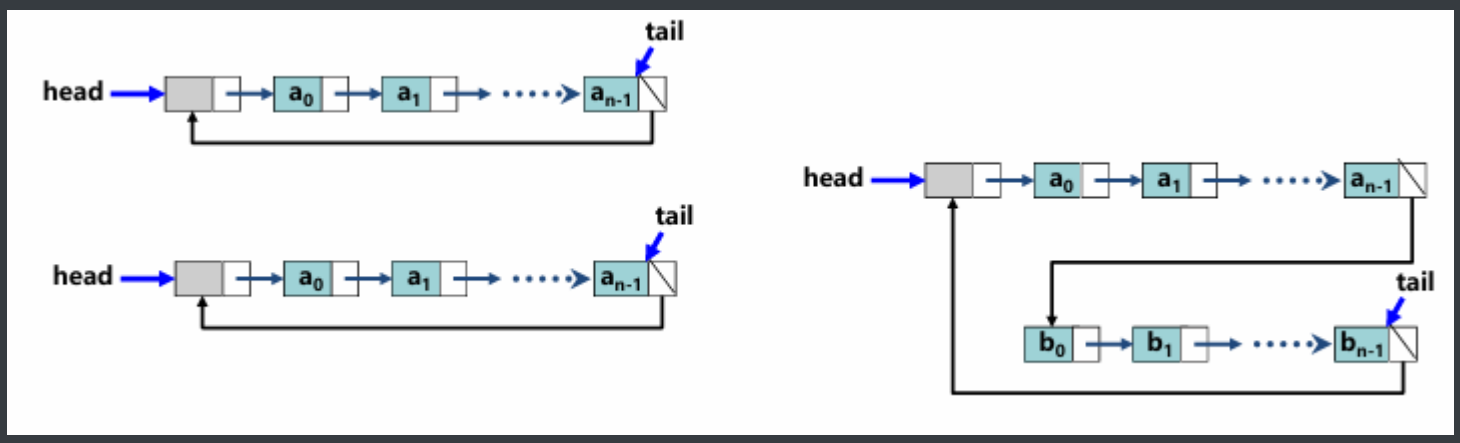
- 简单循环链表



- 带有头结点的单循环链表



如何合并两个单链表?



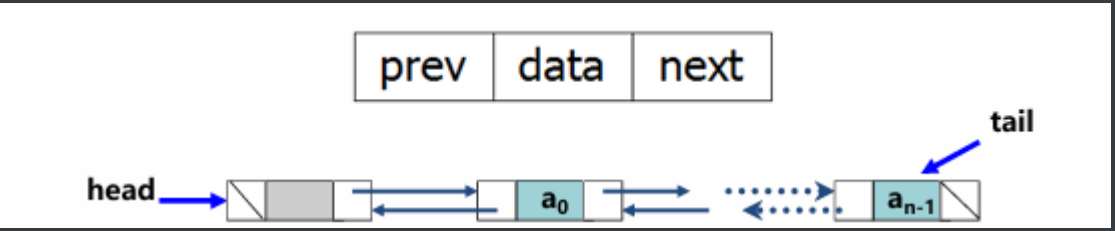
约瑟夫问题的解决算法（单循环链表）

```
class LinkNode():
    def __init__(self,value):
        self.data = value
        self.next = None
n,m = map(int,input().split())
#构建循环链表
def create_circle(n):
    head = LinkNode(1)
    p = head
    for i in range(n-1):
        p.next = LinkNode(i+2)
        p = p.next
    p.next = head
    return head
p = create_circle(n)
while p.next != p:
    #寻找前驱
    for _ in range(m-1):
        prep = p
        p = p.next
    #删除
    prep.next = p.next
    p = p.next
print(p.data)
```

双链表

为了弥补单链表的不足，我们创造出双链表。

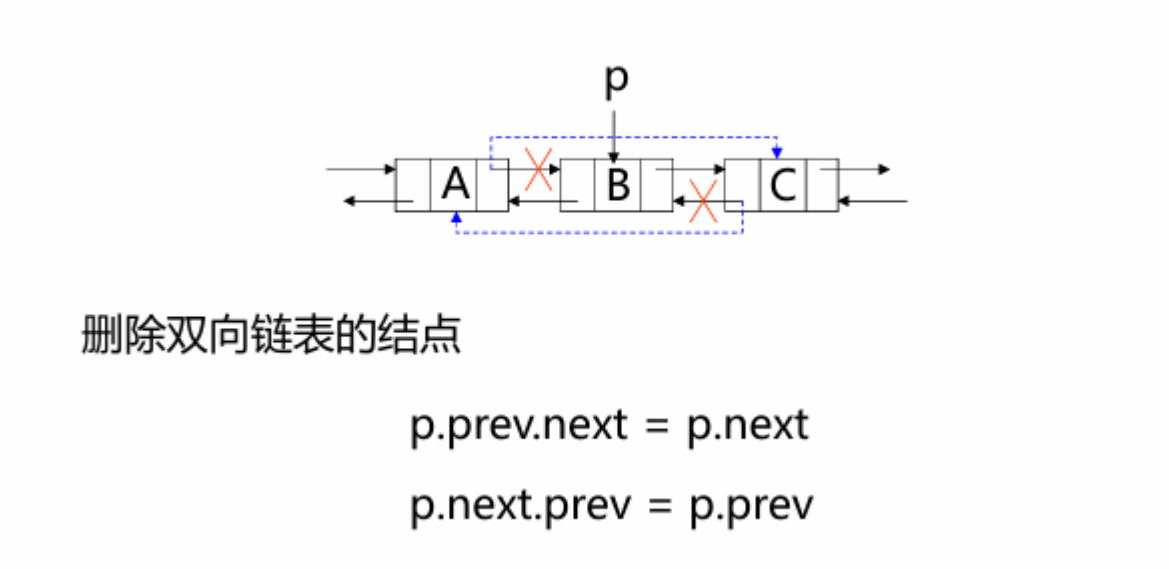
单链表的next字段仅仅指向后继结点，不能有效的找到前驱，反之亦然，因此我们增加一个指向前驱的结点



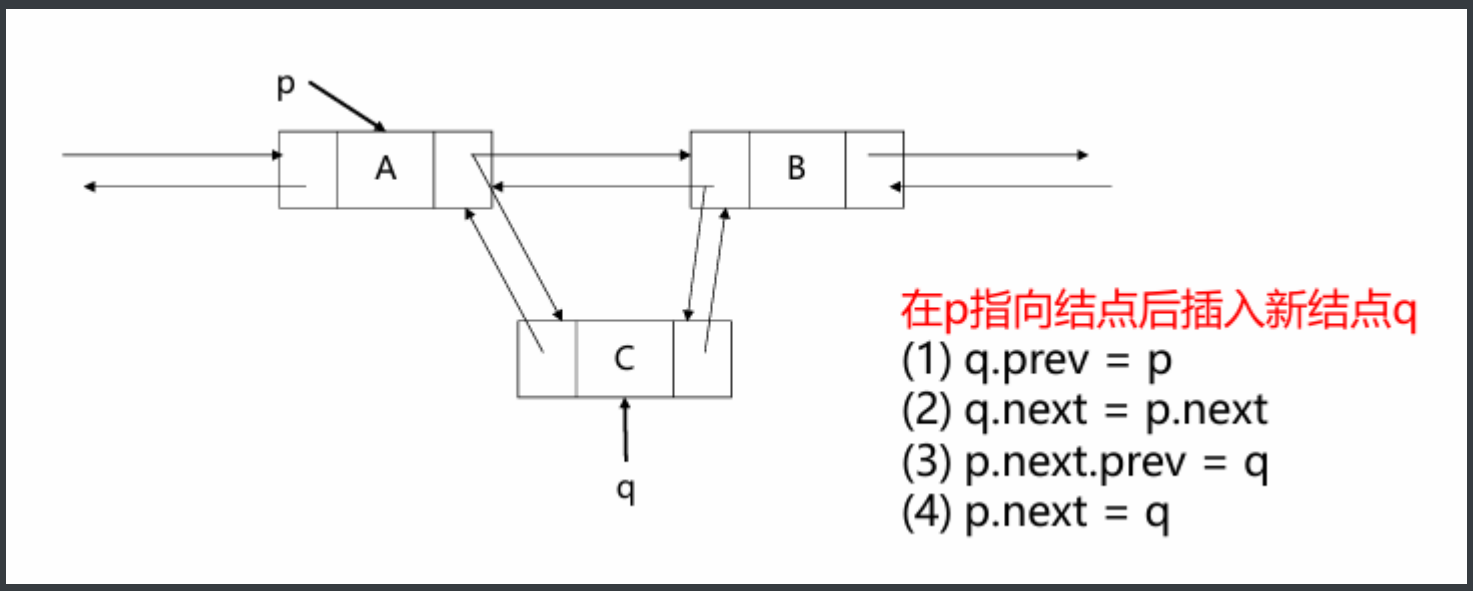
定义

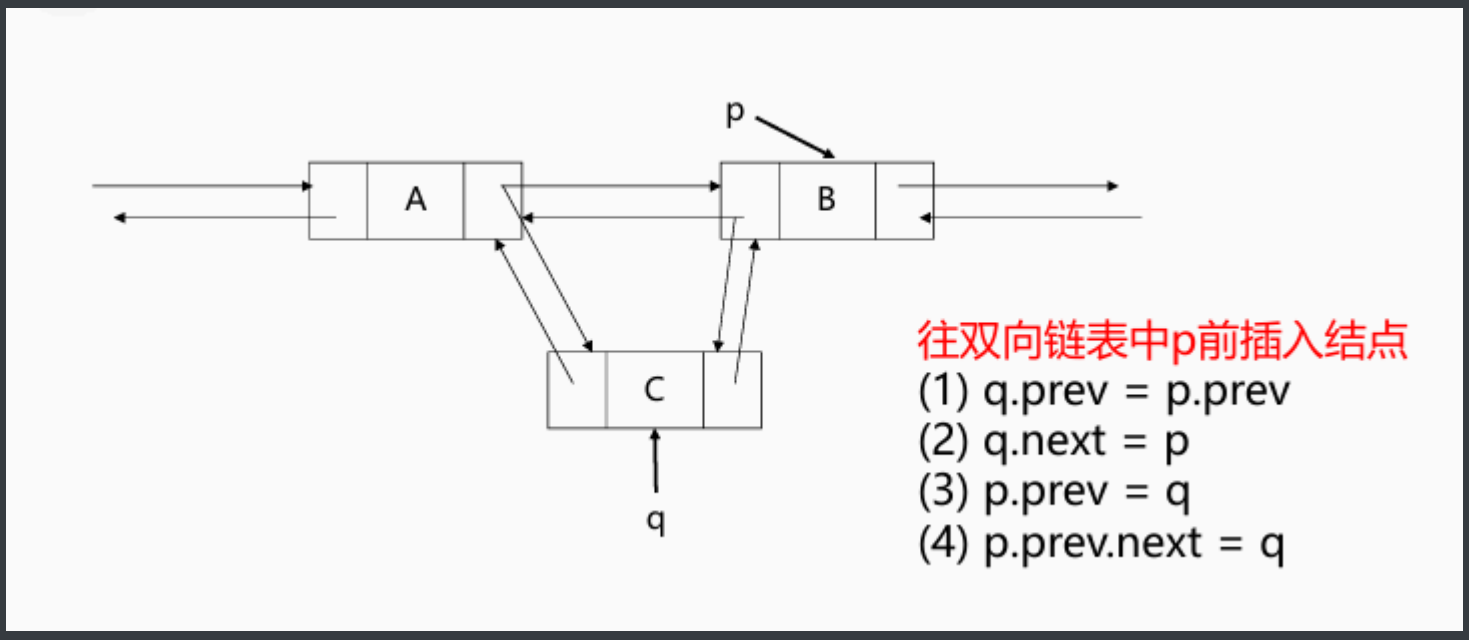
```
class DoubleLinkList():
    def __init__(self):
        self.head = DoubleNode(None)
        self.length = 0
class DoubleNode():
    def __init__(self,node_data):
        self.data = node_data
        self.prev = None
        self.next = None
```

双链表的删除示意图

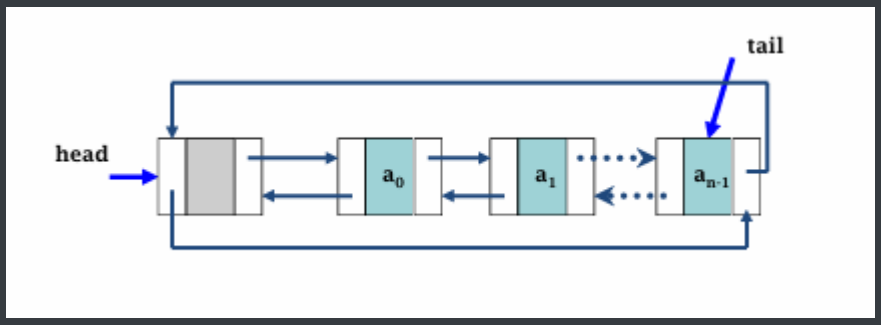


双链表的插入示意图





双向循环链表



链表的边界条件

1. 几个特殊点的处理
    - 头指针处理（**头指针≠头结点**）
    - 非循环链表的尾结点的指针域保持为None
    - 循环链表尾结点的指针回指头结点
2. 链表处理
    - 空链表的特殊处理
    - 插入或删除结点是指针勾链的处理
    - 指针移动的正确性（插入查找或遍历）

思考：有无表头的区别

特点	带表头	不带表头
是否有头结点	是	否
head 指向	头结点	第一个数据节点
第一个节点存数据	不储存	储存
空链表	head.next = None	head = None
插入删除逻辑	统一	需要特殊处理

顺序表、链表的比较

顺序表

- 插入删除O(n)，查找O(1)
- 预先申请固定长度的连续空间
- 如果整个数组元素很满，则没有结构性存储开销
- **优点：不使用指针，读访问便捷，适合存储静态数据**

链表

- 插入删除O(1),查找O(n)
- 存储利用指针，动态的按照需要为表中新元素分配空间

- 每个元素都有结构性开销
- **优点：长度动态变化，适应经常性插入删除内部元素，适合存储动态变化数据**

Python中的线性表

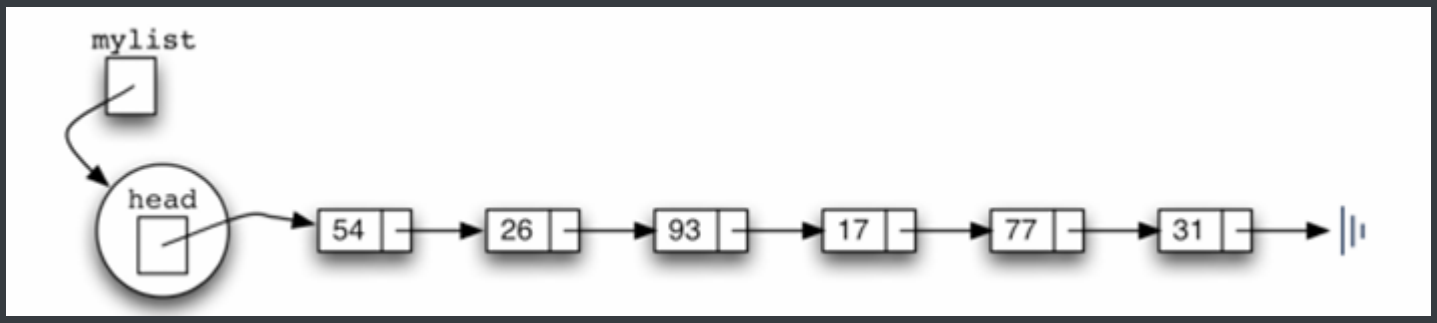
无序表

> 定义：列表是一种数据项按照相对位置存放的数据集，特别的，这种数据集称为 “无序表 unordered list”，其中数据项只按照存放位置来索引，如第1个、 第2个……、最后一个等。

链表实现：结点Node

```
class Node:
    def __init__(self,initdata):
        self.data = initdata
        self.next = None
    def getData(self):
        return self.data
    def getNext(self):
        return self.next
    def setData(self,newdata):
        self.data = newdata
    def setNext(self,newnext):
        self.next = newnext
```

链表实现：无序表



链表实现：add

将新元素加入到链表第一项的代码实现：

```
    def add(self,item):
        temp = Node(item)
        temp.setNext(self.head)
        self.head = temp
```

链表实现：size、search

```
    def size(self):
        current = self.head #注意，这里默认带表头
        count = 0
        while current != None:
            count += 1
            current = current.getNext()
        return count
    def search(self,item):
        current = self.head
        found = False
        while current != None and not found:
            if current.getData() == item:
                found = True
            else:
                current = current.getNext()
        return found
```